UNIVERSIDADE FEDERAL DA BAHIA
INSTITUTO DE COMPUTAÇÃO
DEPARTAMENTO DE CIÊNCIA DA COMPUTAÇÃO

WELLINGTON MIGUEL DE JESUS SILVA

TRABALHO 01 – CRIPTOGRAFIA APLICADA: PROTEÇÃO, ASSINATURAS DIGITAIS E CERTIFICADOS

Estudo de Caso: Auditoria de Contratos Digitais na Miguelito Segurança e Tecnologia Ltda.

SALVADOR - BA
Novembro de 2026

WELLINGTON MIGUEL DE JESUS SILVA

TRABALHO 01 – CRIPTOGRAFIA APLICADA: PROTEÇÃO, ASSINATURAS DIGITAIS E CERTIFICADOS

Estudo de Caso: Auditoria de Contratos Digitais na Miguelito Segurança e Tecnologia Ltda.

Relatório Técnico apresentado à disciplina MATC99 – Segurança e Auditoria de Sistemas de Informação, do Instituto de Computação da Universidade Federal da Bahia (UFBA), como requisito parcial para avaliação prática.

Cenário Auditado: Miguelito Segurança e Tecnologia Ltda.

SALVADOR - BA
Novembro de 2026

SUMÁRIO

1 INTRODUÇÃO E CONTEXTUALIZAÇÃO DO PROBLEMA 3

2 FUNDAMENTAÇÃO TEÓRICA 4

2.1 Criptografia Simétrica vs. Criptografia Assimétrica 4

2.2 Funcionamento do Modo AES-GCM e Integridade Autenticada (AEAD) . . 5

2.3 Infraestrutura de Chaves Públicas (PKI), Certificados X.509 e Assinaturas RSA 7

3 METODOLOGIA E AMBIENTE EXPERIMENTAL 7

4 EXPERIMENTOS DA PARTE I: PROTEÇÃO E INTEGRIDADE COM AES-256-GCM 8

4.1 Procedimentos de Cifragem, Empacotamento e Decifragem 8

4.2 Logs de Execução e Análise dos Cenários de Falha 9

5 EXPERIMENTOS DA PARTE II: ASSINATURAS DIGITAIS E CADEIA DE CERTIFICAÇÃO 10

5.1 Estrutura da PKI Simulada (CA Corporativa vs. Rogue CA) 10

5.2 Análise do Cenário A: Documento Íntegro e Cadeia Válida 10

5.3 Análise do Cenário B: Adulteração de Conteúdo Pós-Assinatura 11

5.4 Análise do Cenário C: Certificado Emitido por Autoridade Não Confiável . 11

6 MATRIZ DE DECISÃO E RECOMENDAÇÕES DE GOVERNANÇA 12

6.1 Matriz Comparativa de Auditoria 12

6.2 Recomendações Técnicas para a Miguelito Segurança e Tecnologia . . . 12

7 CONCLUSÃO 13

8 REFERÊNCIAS BIBLIOGRÁFICAS 14

1 INTRODUÇÃO E CONTEXTUALIZAÇÃO DO PROBLEMA

A digitalização de processos de contratação entre empresas (B2B) substituiu o papel físico e o reconhecimento de firma em cartório por fluxos eletrônicos baseados em rede. Embora essa transição reduza custos operacionais e acelere o fechamento de negócios, ela transfere toda a responsabilidade jurídica e financeira da transação para os controles criptográficos adotados pelas partes. Quando esses controles são mal projetados ou validados apenas de forma parcial, a organização fica exposta a fraudes financeiras e à perda de validade jurídica dos seus instrumentos contratuais.

Neste trabalho, analisa-se o cenário da empresa fictícia **Miguelito Segurança e Tecnologia Ltda.** (CNPJ 14.382.910/0001-55), sediada em Salvador/BA, que atua na operação de sistemas críticos e contrata serviços de nuvem híbrida, links redundantes MPLS/IPSec e custódia de módulos HSM junto a fornecedores externos. Durante uma revisão de auditoria sobre a tramitação do **Contrato Comercial nº 2026/00849-MST**, celebrado com a fornecedora *Parceiro Infraestrutura e Cloud Computing Ltda.* (parceiro.miguelito.com.br, CNPJ 67.419.002/0001-83) no valor global de R$ 1.450.000,00 (dividido em 12 parcelas mensais de R$ 120.833,33), surgiram indícios de duas falhas graves no processo de troca e validação dos arquivos digitais: a suspeita de manipulação de cláusulas financeiras durante o trânsito do documento e o recebimento de contratos assinados com certificados digitais não reconhecidos pela governança interna.

Do ponto de vista de segurança da informação e perícia forense, o incidente enfrentado pela Miguelito Segurança e Tecnologia envolve três riscos práticos distintos que precisam ser isolados e tratados:

a) **interceptação e modificação em trânsito (*Man-in-the-Middle* / *Bit-Flipping*):** contratos comerciais contêm dados bancários, valores de SLA, multas contratuais e especificações técnicas que não podem vazar para terceiros em trânsito (quebra de confidencialidade). Além disso, caso a empresa utilize um algoritmo de cifragem tradicional sem autenticação integrada, um atacante posicionado na rede pode alterar bytes específicos do texto cifrado para modificar valores no texto decifrado sem precisar conhecer a chave secreta;

b) **adulteração documental pós-assinatura:** mesmo quando um contrato é assinado por um representante legítimo, existe o risco de o arquivo sofrer modificações posteriores — seja por ação maliciosa interna, seja por interceptação — alterando, por exemplo, o valor da parcela mensal de R$ 120.833,33 para R$ 920.833,33. O mecanismo de auditoria precisa detectar alterações mínimas de um único caractere e rejeitar o documento

imediatamente;

c) **falsificação de identidade criptográfica (*Key Spoofing via Rogue CA*):** um invasor externo pode gerar seu próprio par de chaves assimétricas, configurar uma Autoridade Certificadora própria (*Rogue CA*) e emitir um certificado digital preenchido com os mesmos dados cadastrais do fornecedor verdadeiro (CN = parceiro.miguelito.com.br). Se o setor responsável pela conferência verificar apenas se a assinatura digital é matematicamente válida contra a chave pública anexada ao processo — esquecendo de verificar quem emitiu aquele certificado —, o contrato fraudulento será aceito como autêntico.

Para responder a esse cenário, este trabalho implementa e documenta uma bancada experimental dividida em duas frentes complementares. Na **Parte I**, utiliza-se a biblioteca cryptography do Python para proteger o contrato com a cifra simétrica autenticada **AES-256-GCM**, testando a recuperação íntegra do arquivo e o comportamento do algoritmo frente à adulteração de bytes no criptograma e ao uso de chaves incorretas. Na **Parte II**, utiliza-se o **OpenSSL** em ambiente Linux para construir uma Infraestrutura de Chaves Públicas (PKI) completa, confrontando a Autoridade Certificadora interna da empresa (Miguelito Root CA) com uma autoridade atacante (Rogue CA) ao longo de três cenários de auditoria (Cenários A, B e C).

2 FUNDAMENTAÇÃO TEÓRICA

2.1 Criptografia Simétrica vs. Criptografia Assimétrica

A escolha entre algoritmos simétricos e assimétricos em uma arquitetura corporativa decorre diretamente das diferenças de desempenho, gestão de chaves e propriedades jurídicas de cada modelo. Na **criptografia simétrica**, o emissor e o destinatário compartilham exatamente a mesma chave secreta *K*, que é usada tanto na operação de cifragem quanto na decifragem (STALLINGS, 2023). Algoritmos simétricos modernos, como o AES (*Advanced Encryption Standard*), operam sobre blocos de bits por meio de rodadas sucessivas de substituição, permutação e combinação linear com chaves derivadas. Por serem operações leves e contarem com instruções dedicadas diretamente nos processadores atuais (*AES-NI*), as cifras simétricas atingem taxas de transferência de múltiplos gigabytes por segundo, sendo a única opção viável na prática para cifrar arquivos extensos, bancos de dados e tráfego de rede em tempo real.

Por outro lado, a criptografia simétrica possui duas limitações importantes para ambientes corporativos abertos: o **problema da distribuição e escalabilidade**

de chaves, visto que em uma rede com *n* empresas parceiras seriam necessárias *n(n-1)/2* chaves secretas distintas trocadas por um canal seguro prévio; e a **ausência de não-repúdio (irretratabilidade)**. Como tanto a contratante quanto a contratada possuem a mesma chave *K*, ambas têm capacidade técnica idêntica para gerar um arquivo cifrado e sua respectiva tag de verificação. Em uma disputa judicial, a Miguelito Segurança e Tecnologia não conseguiria provar a um juiz ou perito externo que determinado arquivo veio exclusivamente do fornecedor, pois a própria contratante poderia tê-lo gerado usando a chave compartilhada.

A **criptografia assimétrica (ou de chave pública)** contorna esses dois limites ao substituir a chave compartilhada por um par de chaves matematicamente vinculadas: uma **chave privada** (mantida em sigilo absoluto pelo seu titular) e uma **chave pública** (que pode ser divulgada abertamente a qualquer parceiro). A segurança desse modelo baseia-se em funções de caminho único com alçapão (*trapdoor one-way functions*), nas quais é fácil calcular a função em um sentido usando a chave pública, mas computacionalmente inviável inverter o cálculo sem conhecer a informação secreta contida na chave privada — como ocorre no problema da fatoração do produto de dois primos grandes no algoritmo RSA (MORIARTY et al., 2016). Quando o titular usa sua chave privada exclusiva para assinar o resumo criptográfico de um documento, qualquer auditor pode usar a chave pública correspondente para conferir a assinatura. Como apenas o titular detém a chave privada, ele não pode negar posteriormente a autoria da assinatura, garantindo o não-repúdio jurídico.

2.2 Funcionamento do Modo AES-GCM e Integridade Autenticada (AEAD)

Durante muitos anos, sistemas corporativos utilizaram cifras de bloco em modos de operação voltados exclusivamente para confidencialidade, como o CBC (*Cipher Block Chaining*) ou o CTR (*Counter Mode*). Embora esses modos escondam o conteúdo do texto em claro contra um observador passivo, eles não oferecem proteção nativa contra um atacante ativo que modifique o texto cifrado em trânsito. No modo Contador (CTR) puro, o algoritmo recebe um valor público inicial único de 96 bits (12 bytes) chamado **Nonce** (IV), concatena a ele um contador sequencial de 32 bits (formando o bloco inicial J_0) e cifra esse contador com a chave secreta *K*. O fluxo pseudoaleatório resultante (*keystream*) é combinado com cada bloco do texto em claro P_i através da operação lógica XOR (□) para produzir o bloco cifrado C_i, conforme expresso na Equação (1):

$$C_i = P_i \square E_K(inc_{32}^{i}(J_0)), \quad i = 1, 2, ..., n \tag{1}$$

O perigo de usar o modo CTR isoladamente reside na sua **maleabilidade linear**: como a decifragem consiste apenas em aplicar novamente a operação XOR entre o bloco cifrado recebido e o mesmo *keystream*, se um invasor interceptar o arquivo no caminho e inverter 1 bit em uma posição conhecida do bloco cifrado C_i, o processo de decifragem não apresentará nenhum erro e inverterá exatamente o mesmo bit no texto em claro recuperado P_i. Em um contrato comercial padronizado onde a posição das cláusulas é previsível, isso permitiria alterar dígitos de valores monetários às cegas (ataque de *bit-flipping*).

Para eliminar essa vulnerabilidade, a norma **NIST SP 800-38D** (DWORKIN, 2007) definiu o **AES-GCM** (*Galois/Counter Mode*), um esquema **AEAD** (*Authenticated Encryption with Associated Data*) que une o modo contador ao mecanismo algébrico de autenticação **GHASH**. Antes de processar os dados, o algoritmo passa um bloco nulo de 128 bits pela cifra AES com a chave K, derivando a subchave secreta de autenticação $H = E_K(0^{128})$. Em seguida, os blocos de 128 bits dos Dados Associados não cifrados (AAD) e os blocos já cifrados ($C_1, ..., C_n$), seguidos de um bloco final com os tamanhos em bits de AAD e do texto cifrado, são tratados como coeficientes de um polinômio avaliado em H dentro do corpo finito de Galois GF(2^{128}) módulo o polinômio irredutível $P(x) = x^{128} + x^7 + x^2 + x + 1$. O resultado de 128 bits do GHASH recebe um XOR final com a cifragem do contador inicial $E_K(J_0)$, gerando a **Authentication Tag (T)** de 16 bytes (128 bits), descrita na Equação (2):

$$T = GHASH_H(AAD, Ciphertext) \oplus E_K(J_0) \quad (2)$$

O diferencial de segurança do AES-GCM ocorre na decifragem, que segue estritamente a regra **Encrypt-then-MAC**: antes de liberar qualquer byte decifrado para a aplicação, a rotina recalcula a tag local T' sobre os blocos recebidos e a compara em tempo constante com a tag T anexada ao arquivo. Se um atacante alterar qualquer bloco cifrado C_i por uma máscara de erro não nula Δ (ou seja, $C'_i = C_i \oplus \Delta$), a diferença algébrica entre a tag recalculada e a original em GF(2^{128}) é dada pela Equação (3):

$$\Delta T = T' \oplus T = \Delta \cdot H^{n-i+2} \pmod{P(x)} \quad (3)$$

Como GF(2^{128}) é um corpo finito e não admite divisores de zero, o produto de dois elementos não nulos ($\Delta \neq 0$ e $H \neq 0$) jamais resulta em zero. Portanto, $T' \neq T$ obrigatoriamente, fazendo com que a biblioteca descarte o buffer de saída e lance imediatamente a exceção InvalidTag.

2.3 Infraestrutura de Chaves Públicas (PKI), Certificados X.509 e Assinaturas RSA

Embora o AES-256-GCM proteja o arquivo em trânsito e em repouso entre sistemas que já compartilham uma chave, ele não substitui a assinatura jurídica de um contrato entre empresas distintas. Para garantir que a fornecedora não possa repudiar o contrato assinado, emprega-se a assinatura digital **RSA com SHA-256** segundo o paradigma **Hash-then-Sign** (MORIARTY et al., 2016). Primeiro, o documento passa pela função hash **SHA-256** (NIST, 2015), que comprime o arquivo em um resumo fixo de 256 bits (32 bytes) dotado de efeito avalanche e resistência a colisões. No esquema **RSASSA-PKCS1-v1_5** (ou de modo probabilístico no **RSA-PSS**), o hash de 32 bytes e o identificador (OID) do algoritmo SHA-256 são encapsulados em uma estrutura ASN.1 (DigestInfo) com preenchimento até o tamanho do módulo RSA (256 bytes para chaves de 2048 bits), formando o bloco codificado *EM*. A geração da assinatura *S* com a chave privada *(d, N)* e a sua verificação com a chave pública *(e, N)* seguem a Equação (4):

$$S = EM^{d} \ (mod\ N) \quad e \quad EM' = S^{e} \ (mod\ N) \tag{4}$$

Todavia, a operação matemática da Equação (4) valida apenas que a assinatura foi produzida pelo detentor da chave privada correspondente àquela chave pública, mas não prova a quem pertence essa chave pública. Para vincular formalmente uma identidade jurídica a uma chave pública, utiliza-se a **Infraestrutura de Chaves Públicas (PKI)** e os certificados **X.509 v3** padronizados pela **RFC 5280** (COOPER et al., 2008). A organização institui uma **Autoridade Certificadora Raiz (Root CA)** — neste estudo, a Miguelito Root CA — cujo certificado autoassinado é instalado na **Trust Store** corporativa. A Root CA assina digitalmente o certificado do fornecedor (parceiro.crt), amarrando seu nome (Subject: CN = parceiro.miguelito.com.br) à sua chave pública e fixando extensões críticas (basicConstraints: CA:FALSE e keyUsage: digitalSignature, nonRepudiation). Dessa forma, a auditoria de um contrato assinado exige sempre duas checagens conjuntas: a verificação matemática da assinatura e a validação da cadeia X.509 até a Root CA corporativa (incluindo checagem de revogação via CRL ou OCSP).

3 METODOLOGIA E AMBIENTE EXPERIMENTAL

Para validar experimentalmente os controles criptográficos exigidos pela Miguelito Segurança e Tecnologia, foram construídos dois scripts automatizados em ambiente Linux (Ubuntu 24.04 LTS sob WSL2), registrando todos os eventos, resumos SHA-256 e códigos de retorno em arquivos de log. A Tabela 1 detalha a especificação do ambiente experimental.

Tabela 1 – Especificação das ferramentas e ambiente experimental da auditoria

Componente / Ferramenta	Versão / Especificação	Finalidade nos Experimentos
Sistema Operacional	Ubuntu 24.04 LTS (Kernel 6.6.87.2 WSL2)	Ambiente POSIX isolado para execução dos scripts e coleta de logs.
Interpretador Python	Python 3.12.3 / 3.13.14	Execução do módulo parte1_aes_gcm.py com logging e type hints.
Biblioteca Criptográfica	cryptography 41.0.7 / 50.0.2 (AESGCM)	Cifragem autenticada AES-256-GCM e captura de InvalidTag.
Suíte OpenSSL CLI	OpenSSL 3.0.13 (30 Jan 2024)	Geração de chaves RSA 4096/2048-bit, emissão X.509 v3, dgst e verify.
Contrato Comercial Base	contrato_miguelito.txt (2.841 bytes)	Contrato nº 2026/00849-MST (R$ 1.450.000,00 / 12x R$ 120.833,33).

Fonte: O autor (2026).

4 EXPERIMENTOS DA PARTE I: PROTEÇÃO E INTEGRIDADE COM AES-256-GCM

4.1 Procedimentos de Cifragem, Empacotamento e Decifragem

O script parte1_aes_gcm.py encapsula os artefatos criptográficos na classe CriptogramaGCM e executa inicialmente o fluxo legítimo em três etapas. Na primeira etapa, o contrato comercial contrato_miguelito.txt é lido em modo binário, totalizando exatos **2.841 bytes** e apresentando o resumo SHA-256 de referência 9147c03987856361e26a46f794f9588fe0850329cc2eaa28d5f0fd6ffe0cfe67.

Na segunda etapa, gera-se uma chave simétrica de 256 bits (32 bytes) via AESGCM.generate_key(256), um Nonce aleatório de 96 bits (12 bytes) via os.urandom(12) e define-se o campo de Dados Associados (AAD) como MATC99-UFBA|MIGUELITO-SEGURANCA-E-TECNOLOGIA|CONTRATO-2026-00849. O criptograma completo é persistido no arquivo contrato_miguelito.enc com **2.869 bytes** (12 bytes de Nonce + 2.841 bytes de Ciphertext + 16 bytes de Authentication Tag). Na terceira etapa, o script lê o arquivo .enc do disco, valida a Tag de 128 bits no método AESGCM.decrypt() e grava o texto recuperado em contrato_miguelito_recuperado.txt, comprovando igualdade estrita de bytes (original == recuperado: True) e preservação integral do hash SHA-256.

4.2 Logs de Execução e Análise dos Cenários de Falha

Concluída a validação legítima, o script submeteu o criptograma aos dois cenários de falha exigidos pela especificação: adulteração intencional de 1 byte no texto cifrado (*bit-flipping*) e tentativa de decifragem com chave simétrica incorreta. O Quadro 1 apresenta o log de execução capturado.

Quadro 1 – Log de execução da Parte I (AES-256-GCM e captura de InvalidTag)

```
[2026-11-04 15:36:36.889] [INFO] === Auditoria Criptográfica - Miguelito Segurança e Tecnologia (Parte I: AES-256-GCM) ===
[2026-11-04 15:36:36.894] [INFO] Contrato carregado: contrato_miguelito.txt (2841 bytes)
[2026-11-04 15:36:36.895] [INFO] SHA-256 original: 9147c03987856361e26a46f794f9588fe0850329cc2eaa28d5f0fd6ffe0cfe67
[2026-11-04 15:36:36.901] [INFO] Arquivo cifrado salvo em: contrato_miguelito.enc (2869 bytes totais)
[2026-11-04 15:36:36.901] [INFO]   Chave AES-256 (32B) : 185c3c58dc5153a9b84ddaf0b85ec9933e52f67c44f8d150f42151a460aa8a28
[2026-11-04 15:36:36.902] [INFO]   Nonce / IV (12B)    : 1d5d2cb6580417d1f29dbf07
[2026-11-04 15:36:36.902] [INFO]   AAD                 : MATC99-UFBA|MIGUELITO-SEGURANCA-E-TECNOLOGIA|CONTRATO-2026-00849
[2026-11-04 15:36:36.903] [INFO]   Auth Tag (16B)      : f987ee9b9f25a4ea8cbcac904a675b94
[2026-11-04 15:36:36.909] [INFO] Comparação de bytes (original == recuperado): True (2841 bytes)
[2026-11-04 15:36:36.910] [INFO] --- [Teste de Falha 1: Adulteração de 1 byte no Ciphertext (Bit-Flipping)] ---
[2026-11-04 15:36:36.910] [INFO] Byte modificado no offset 1420 (0x058C): 0x4B -> 0x4A
[2026-11-04 15:36:36.910] [WARNING] Exceção capturada com sucesso: cryptography.exceptions.InvalidTag
[2026-11-04 15:36:36.911] [INFO] Motivo: A alteração no bloco cifrado mudou o resultado do polinômio GHASH em GF(2^128).
[2026-11-04 15:36:36.911] [INFO] --- [Teste de Falha 2: Tentativa de Decifragem com Chave Incorreta] ---
[2026-11-04 15:36:36.911] [INFO] Chave correta : 185c3c58dc5153a9b84ddaf0b85ec9933e52f67c44f8d150f42151a460aa8a28
[2026-11-04 15:36:36.912] [INFO] Chave errada  : 8908f8e494c7d2089963f9ca67836a1d43bfc0a0ea511d3343b5c0a40d8df58e
[2026-11-04 15:36:36.912] [WARNING] Exceção capturada com sucesso: cryptography.exceptions.InvalidTag
```

Fonte: O autor (2026).

Conforme registrado no Quadro 1, no **Teste de Falha 1** foi invertido 1 bit no byte localizado no offset 1420 (0x058C, alterando 0x4B para 0x4A via XOR 0x01). Durante a tentativa de decifragem com a chave correta, a avaliação do polinômio GHASH em GF(2^{128}) resultou em uma tag divergente de f987ee9b..., disparando imediatamente a exceção cryptography.exceptions.InvalidTag sem liberar o texto em claro. De forma análoga, no **Teste de Falha 2**, a utilização da chave incorreta 8908f8e4... produziu uma subchave de hash *H′* e uma máscara de contador distintas das originais, bloqueando a operação com InvalidTag. Comprovam-se, assim, as propriedades de confidencialidade, integridade e autenticidade simétrica do criptograma.

5 EXPERIMENTOS DA PARTE II: ASSINATURAS DIGITAIS E CADEIA DE CERTIFICAÇÃO

5.1 Estrutura da PKI Simulada (CA Corporativa vs. Rogue CA)

O script parte2_pki_testes.sh provisionou no diretório pki_miguelito/ duas hierarquias de certificação X.509 v3 independentes. Na **Cadeia Corporativa Legítima**, foi gerada a autoridade raiz interna Miguelito Root CA com chave RSA de 4096 bits e certificado autoassinado (miguelito_root_ca.crt, Serial 362E0342...2D96, basicConstraints: CA:true), a qual emitiu e assinou o certificado legítimo do fornecedor parceiro.crt (CN = parceiro.miguelito.com.br, RSA 2048 bits, Serial 21CFB1F0...BCC2, CA:FALSE). Paralelamente, na **Cadeia Atacante**, configurou-se uma autoridade externa não confiável (Rogue Root CA, RSA 4096 bits) que emitiu um certificado forjado (parceiro_forjado.crt, Serial 35D9F02A...7127) preenchido com o mesmo CN = parceiro.miguelito.com.br do fornecedor legítimo.

5.2 Análise do Cenário A: Documento Íntegro e Cadeia Válida

No Cenário A, o documento original foi assinado com a chave privada legítima do fornecedor e auditado em duas etapas: verificação matemática da assinatura contra a chave pública do certificado e validação da cadeia X.509 contra a Miguelito Root CA, conforme detalhado no Quadro 2.

Quadro 2 – Comandos e log de verificação do Cenário A (Assinatura e Cadeia Legítimas)

```
# Comandos executados no Cenário A:
$ openssl dgst -sha256 -sign pki_miguelito/parceiro_legitimo/parceiro.key -out cenario_a.sig contrato_miguelito.txt
$ openssl dgst -sha256 -verify pki_miguelito/parceiro_legitimo/parceiro_pub.pem -signature cenario_a.sig contrato_miguelito.
$ openssl verify -CAfile pki_miguelito/root_ca/miguelito_root_ca.crt pki_miguelito/parceiro_legitimo/parceiro.crt

# Evidências coletadas em parte2_pki_testes.log:
[2026-11-04 15:36:40.697] [INFO] Assinatura gerada: cenario_a.sig (256 bytes)
[2026-11-04 15:36:40.721] [INFO] Verificação da assinatura (openssl dgst -verify): Verified OK
[2026-11-04 15:36:40.743] [INFO] Validação da cadeia contra Miguelito Root CA: pki_miguelito/parceiro_legitimo/parceiro.crt:
[2026-11-04 15:36:40.750] [INFO] Resultado Cenário A -> DECISÃO: ACEITAR
```

Fonte: O autor (2026).

O resultado Verified OK comprova a integridade matemática do contrato após a assinatura, enquanto parceiro.crt: OK atesta que o certificado do signatário pertence à cadeia de confiança da Miguelito Root CA, justificando a decisão de **ACEITAR** o documento.

5.3 Análise do Cenário B: Adulteração de Conteúdo Pós-Assinatura

No Cenário B, simulou-se a adulteração de 1 caractere no valor da parcela mensal do contrato após a assinatura (de R$ 120.833,33 para R$ 920.833,33), submetendo o documento modificado à verificação contra a assinatura original colhida no Cenário A. O Quadro 3 exibe os resultados obtidos.

Quadro 3 – Comandos e log de verificação do Cenário B (Adulteração Pós-Assinatura)

```
# Comandos executados no Cenário B:
$ sed 's/R\$ 120\.833,33/R\$ 920.833,33/g' contrato_miguelito.txt > contrato_adulterado.txt
$ openssl dgst -sha256 -verify pki_miguelito/parceiro_legitimo/parceiro_pub.pem -signature cenario_a.sig contrato_adulterado

# Evidências coletadas em parte2_pki_testes.log:
[2026-11-04 15:36:40.824] [INFO] SHA-256 original   : 9147c03987856361e26a46f794f9588fe0850329cc2eaa28d5f0fd6ffe0cfe67
[2026-11-04 15:36:40.832] [INFO] SHA-256 adulterado : 817d7fae7346f151320d2469a8546c52edefa478219948e1efbb75edf9f0a8d2
[2026-11-04 15:36:40.856] [WARN] Código de saída do OpenSSL: 1
  40C71E4E407F0000:error:02000068:rsa routines:ossl_rsa_verify:bad signature:../crypto/rsa/rsa_sign.c:430:
  Verification failure
[2026-11-04 15:36:40.868] [WARN] Resultado Cenário B -> DECISÃO: REJEITAR
```

Fonte: O autor (2026).

Conforme demonstrado no Quadro 3, a alteração de um único dígito provocou o efeito avalanche na função SHA-256, mudando o resumo de 9147c039... para 817d7fae.... Como o hash extraído do bloco PKCS#1 v1.5 da assinatura não confere com o novo hash calculado, a rotina ossl_rsa_verify falhou com bad signature / Verification failure (Exit Code 1), determinando a decisão de **REJEITAR** o contrato.

5.4 Análise do Cenário C: Certificado Emitido por Autoridade Não Confiável

No Cenário C, o documento original íntegro foi assinado com a chave privada do certificado falso (parceiro_forjado.key) emitido pela Rogue CA. O Quadro 4 apresenta o contraste entre a verificação matemática isolada e a validação da cadeia de certificação.

Quadro 4 – Comandos e log de verificação do Cenário C (Certificado Forjado por Rogue CA)

```
# Comandos executados no Cenário C:
$ openssl dgst -sha256 -sign pki_miguelito/parceiro_forjado/parceiro_forjado.key -out cenario_c_forjado.sig contrato_migueli
$ openssl dgst -sha256 -verify pki_miguelito/parceiro_forjado/parceiro_forjado_pub.pem -signature cenario_c_forjado.sig cont
$ openssl verify -CAfile pki_miguelito/root_ca/miguelito_root_ca.crt pki_miguelito/parceiro_forjado/parceiro_forjado.crt

# Evidências coletadas em parte2_pki_testes.log:
[2026-11-04 15:36:40.927] [INFO] 1) Verificação matemática com a chave pública do certificado falso: Verified OK
[2026-11-04 15:36:40.949] [WARN] 2) Validação da cadeia contra a Trust Store da Miguelito (Exit Code 2):
  C = BR, ST = Sao Paulo, L = Sao Paulo, O = Parceiro Infraestrutura e Cloud Computing Ltda, OU = Contratos Digitais B2B, CN
  error 20 at 0 depth lookup: unable to get local issuer certificate
  error pki_miguelito/parceiro_forjado/parceiro_forjado.crt: verification failed
[2026-11-04 15:36:40.962] [WARN] Resultado Cenário C -> DECISÃO: REJEITAR
```

Fonte: O autor (2026).

A análise do Quadro 4 evidencia a diferença crítica entre integridade matemática e autenticidade da origem: a primeira verificação retorna Verified OK porque a assinatura foi conferida contra a própria chave pública falsa que a gerou. Todavia, na segunda etapa (openssl verify), a validação falha no nível 0 da cadeia (error 20 at 0 depth lookup: unable to get local issuer certificate, Exit Code 2), pois a Rogue Root CA não pertence à *Trust Store* da Miguelito Segurança e Tecnologia, impondo a decisão pericial de **REJEITAR** o documento.

6 MATRIZ DE DECISÃO E RECOMENDAÇÕES DE GOVERNANÇA

6.1 Matriz Comparativa de Auditoria

A Tabela 2 sintetiza os resultados obtidos nos experimentos das Partes I e II, relacionando a integridade matemática do arquivo, a confiabilidade da identidade/chave e o veredito final de auditoria.

Tabela 2 – Matriz comparativa de decisão dos cenários auditados nas Partes I e II

Cenário Auditado	Integridade Matemática	Confiabilidade da Chave / CA	Evidência Técnica Capturada	Veredito
Parte I – Fluxo Normal	Válida (Tag GCM e SHA-256 OK)	Válida (Chave simétrica correta)	2.841 bytes recuperados sem erro	**ACEITAR**
Parte I – Bit-Flipping	Inválida (GHASH diverge)	Válida (Chave simétrica correta)	cryptography.exceptions. InvalidTag	**REJEITAR**
Parte I – Chave Espúria	Não verificável (H' ≠ H)	Inválida (Chave não autorizada)	cryptography.exceptions. InvalidTag	**REJEITAR**
Parte II – Cenário A	Válida (Verified OK)	Válida (Miguelito Root CA: OK)	Exit Code 0 em dgst e verify	**ACEITAR**
Parte II – Cenário B	Inválida (Verification failure)	Válida (Certificado legítimo)	ossl_rsa_verify:bad signature (Exit 1)	**REJEITAR**
Parte II – Cenário C	Válida (Verified OK isolado)	Inválida (Fora da Trust Store)	error 20: unable to get issuer (Exit 2)	**REJEITAR**

Fonte: O autor (2026).

6.2 Recomendações Técnicas para a Miguelito Segurança e Tecnologia

Com base nos achados periciais, recomendam-se quatro medidas técnicas para a governança criptográfica da Miguelito Segurança e Tecnologia Ltda.:

a) **validação obrigatória em duas etapas e fixação da Trust Store:** o fluxo automatizado de recebimento de contratos deve executar obrigatoriamente a validação da cadeia X.509 (openssl verify -CAfile miguelito_root_ca.crt) antes

de conferir a assinatura RSA, bloqueando certificados emitidos por CAs externas não homologadas;

b) **isolamento da Root CA em HSM e adoção de CA Intermediária:** manter a chave privada RSA de 4096 bits da Miguelito Root CA isolada *offline* em módulo HSM (FIPS 140-3) e instituir uma Autoridade Certificadora Intermediária para emitir os certificados operacionais dos parceiros B2B;

c) **verificação de revogação (CRL / OCSP) e restrição de uso de chave:** publicar Listas de Certificados Revogados (CRL) e exigir consulta OCSP (-crl_check) para barrar chaves privadas comprometidas antes da expiração, além de validar a presença de CA:FALSE e keyUsage: digitalSignature, nonRepudiation;

d) **gerenciamento seguro de Nonces e rotação de chaves no AES-256-GCM:** garantir que cada operação de cifragem utilize um Nonce de 12 bytes exclusivo gerado via CSPRNG e rotacionar periodicamente as chaves simétricas, prevenindo ataques de reutilização de Nonce que exporiam o *keystream* e a subchave *H* do GHASH.

7 CONCLUSÃO

O trabalho experimental desenvolvido para o cenário da **Miguelito Segurança e Tecnologia Ltda.** permitiu demonstrar na prática como a cifragem simétrica autenticada e a infraestrutura de chaves públicas atuam de forma complementar na proteção de contratos digitais. Nos testes da Parte I, comprovou-se que o modo **AES-256-GCM** elimina a maleabilidade linear do modo contador puro por meio do polinômio **GHASH** em GF(2^{128}), bloqueando instantaneamente tanto a adulteração de 1 bit no texto cifrado quanto a tentativa de leitura com chave espúria através da exceção InvalidTag.

Nos experimentos da Parte II, a implementação da PKI com o OpenSSL evidenciou a fronteira entre integridade de conteúdo e autenticidade da identidade do signatário. Enquanto o Cenário B comprovou que qualquer alteração pós-assinatura em cláusulas financeiras invalida o resumo **SHA-256** (Verification failure), o Cenário C demonstrou que uma assinatura matematicamente válida (Verified OK) deve ser rejeitada quando o certificado que transporta a chave pública não possui vínculo de confiança com a Miguelito Root CA (error 20 at 0 depth lookup: unable to get local issuer certificate). Conclui-se que a combinação do transporte cifrado em AES-256-GCM com a validação estrita da cadeia X.509 assegura integralmente a confidencialidade, a integridade, a autenticidade e o não-repúdio exigidos pela auditoria.

8 REFERÊNCIAS BIBLIOGRÁFICAS

COOPER, D.; SANTESSON, S.; FARRELL, S.; BOEYEN, S.; HOUSLEY, R.; POLK, W. **Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile**. [S. l.]: Internet Engineering Task Force (IETF), RFC 5280, maio 2008. Disponível em: <https://datatracker.ietf.org/doc/html/rfc5280>. Acesso em: 04 nov. 2026.

DWORKIN, Morris J. **Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC**. Gaithersburg, MD: National Institute of Standards and Technology (NIST), Special Publication 800-38D, nov. 2007.

MORIARTY, K.; KALISKI, B.; JONSSON, J.; RUSCH, A. **PKCS #1: RSA Cryptography Specifications Version 2.2**. [S. l.]: Internet Engineering Task Force (IETF), RFC 8017, nov. 2016. Disponível em: <https://datatracker.ietf.org/doc/html/rfc8017>. Acesso em: 04 nov. 2026.

NATIONAL INSTITUTE OF STANDARDS AND TECHNOLOGY (NIST). **FIPS PUB 180-4: Secure Hash Standard (SHS)**. Gaithersburg, MD: U.S. Department of Commerce, ago. 2015.

PYTHON CRYPTOGRAPHIC AUTHORITY (PyCA). **Cryptography Documentation: Authenticated Encryption with Associated Data (AESGCM)**. [S. l.]: PyCA, 2026. Disponível em: <https://cryptography.io/en/latest/hazmat/primitives/aead/>. Acesso em: 04 nov. 2026.

STALLINGS, William. **Criptografia e segurança de redes: princípios e práticas**. 8. ed. São Paulo: Pearson Education, 2023.